Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques

Neural Lossless Coding

Conclusions

Neural Lossless Coding: The Density Estimation Perspective

Neural Network (NN) estimates the true, high-order distribution $P(X^n)$ of the source

The Information-Theoretic Limit

In Neural Lossless Coding (NLC), we minimize the **Cross-Entropy** between the true distribution P and the neural model (estimated distribution) Q:

$$H(P,Q) = H(P) + D_{KL}(P \parallel Q)$$

- $H(P)$ is the source entropy (physical limit).
- D_{KL} is the penalty for model inaccuracy (redundancy).
- The NN does not "compress" directly, but acts as a **probability distribution estimator** providing Q to an arithmetic coder.

Overcoming the Curse of Dimensionality

- **Traditional Approach**: State-space explodes exponentially (M^N). A 3-pixel context at 8 bpp requires ≈ 16.7 million entries .
- **Neural Approach**: Maps contexts to a continuous vector space. It learns to group similar patterns, generalizing even to unseen contexts.
- **Efficiency**: While classical context-coding reached 0.406 bpp for the T-shape, NLC aims for the true entropy rate $\mathcal{H}$ by leveraging global dependencies (Attention/Transformers).

Key takeaway: NNs replace look-up tables with non-linear function approximation.

1. Autoregressive Models: Causal Density Estimation

Mathematical Foundation

These models treat the image as a sequence of random variables and apply the **Chain Rule** of probability:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1})$$

- **Working Principle**: The network estimates the conditional PDF of the current pixel x_i given all previous pixels (causal context).
- **Information Theory**: Since the model captures high-order dependencies, the bit-rate approaches the **Entropy Rate** $\mathcal{H}$.
- **Trade-off**:

2. Latent Variable Models: Mapping to Manifolds

Theoretical Framework

Based on **Variational Inference**. The image x is mapped to a lower-dimensional latent representation z (similar to a non-linear PCA).

- **Two-stage Coding**:
  1. Encode the latent features z (the "summary").
  2. Encode the data x given z (the "residuals" or likelihood).
- **The Bits-back Principle**: In a sequence of images, the bits used to specify z can be "recovered" from the randomness of the previous message.
- **Limit**: The compression performance is bounded by the **ELBO** (Evidence Lower Bound), which is the variational approximation of the true log-likelihood.

3. Neural Predictive Coding: Non-linear Prediction

Evolution of DPCM

This approach enhances the classical Differential Pulse Code Modulation by replacing the linear predictor with a non-linear Neural Network.

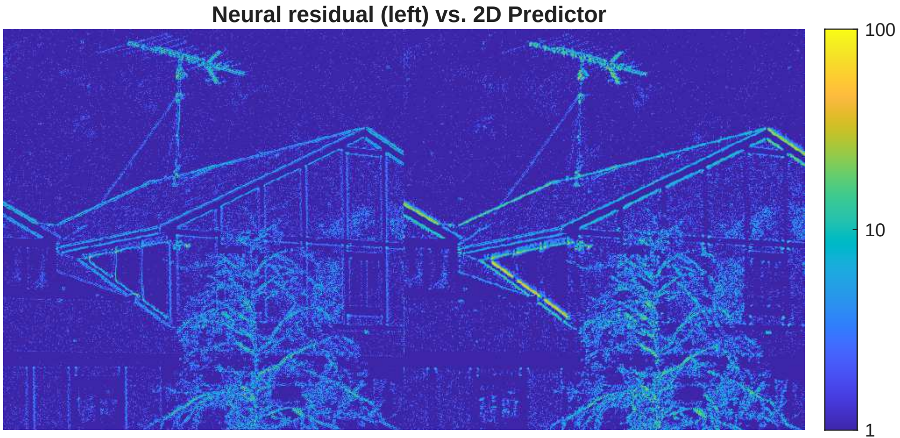
- **Concept**: The prediction $\hat{x}_i = f_{NN}(\text{Local Context})$ is no longer a fixed weighted average, but a learned function that adapts to complex edges and textures.
- **Loss Function**: Unlike standard NNs (trained on MSE), these are trained to minimize the **Cross-Entropy** of the residual distribution.
- **Implementation**: High throughput compared to AR models, as it can be easily integrated into existing frameworks like JPEG-LS or PNG.

Neural Predictor Architecture

In this experiment a **Multi-Layer Perceptron (MLP)** has been used:

- **Input**: Local causal neighborhood $S = \{A, B, C\}$.
- **Function**: $\hat{Y} = f_{MLP}(S, \mathrm{W})$, where W are the learned weights.
- **Final Code**: The prediction is rounded to ensure integer residuals.

Experimental Results: Residual Analysis

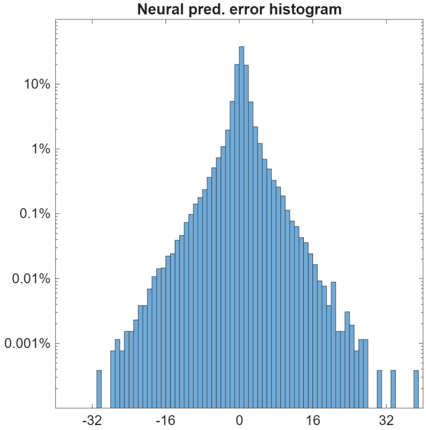


Experimental Results: Residual Analysis

Neural vs. 2D Predictor

- **Left (Neural)**: The residual map is noticeably "cleaner" and more uniform.
- **Right (2D)**: Shows stronger residuals (yellow/green) especially on diagonal structures and the antenna.
- **Key Observation**: The NN better handles complex textures and non-linear transitions that the deterministic logic ($|C - B| < |C - A|$) misses.

Residual Histogram & Entropy



Statistical Properties

- **Distribution**: The neural residuals follow a highly peaked **Laplacian distribution** centered at zero.
- **Sparsity**: The histogram shows that the NN successfully maps the image to a sparse representation.
- **Entropy Gain**: By narrowing the error distribution, the NN reduces the zero-order entropy of residuals, from 2.830 to 2.704 bpp.

Counting the Bitrate Overhead

The total cost to transmit the image is:

$$R_{total} = R_{residuals} + \frac{L_{model}}{N} \quad \text{[bits/pixel]}$$

- $L_{model} = \text{(Total Parameters)} \times \text{(Precision)}$.
- **Example**: A small [10, 5] MLP with 3 inputs has ≈ 100 weights. Using 32-bit floats, $L_{model} = 3200$ bits.
- Total rate: $R = 2.704 + \frac{3200}{512^2} \approx 2.716$
- **Trade-off**: For small images (N), the model cost can outweigh the compression gain. NLC is more efficient for large datasets (weight shared among all the images) or high-resolution images.

Current Limitations of Neural Lossless Coding

Despite superior compression ratios, NLC faces significant deployment hurdles.

- **Computational Complexity**: AR models require N forward passes for an N-pixel image, making real-time decoding nearly impossible without massive optimization.
- **Generalization Gap**: NNs are sensitive to "out-of-distribution" data. A model trained on natural images may fail (or expand) medical or satellite imagery.
- **Hardware Requirements**: Unlike Huffman or LZW, NLC requires GPUs or specialized AI accelerators (NPUs) at the decoder side.
- **Deterministic Requirements**: Lossless coding requires perfect bit-reproducibility across different hardware (handling floating-point rounding errors).

SOTA Comparison: Neural vs. Traditional Lossless

Performance comparison on standard datasets (e.g., Kodak). Values in **bits per pixel (bpp)**: lower is better.

Codec	Type	Avg. bpp	Decoding Complexity
PNG (Deflate)	Traditional	4.30 – 4.50	Very Low
WebP (Lossless)	Traditional	3.70 – 3.90	Low
JPEG-LS	Traditional	3.60 – 3.80	Low
L3C (CNN)	Neural (AR)	3.30 – 3.50	Very High
MS-ILLC	Neural (Hier.)	3.10 – 3.30	High
Transformer-based	Neural (Att.)	**2.90 – 3.10**	Ultra High

- **The Efficiency Gap**: Neural models provide a **15% to 30%** improvement over PNG.
- **Contextual Power**: Transformers can identify repeating textures or semantic patterns

Outline

Comparison of Lossless Coding Techniques

Method	Efficiency	Complexity	Latency	Best use case
Neural (SOTA)	Highest	Ultra High	High	SOTA Image/Video
Arithmetic	High (to H)	High (CPU)	Med	Bi-level images, CABAC
Dictionary	Universal	Moderate	Low/Med	Repeating patterns
Huffman	Good to high ($1 < L < H + 1$)	Low/Med	Very Low	JPEG, general purpose
Exp-Golomb	Good (approx.)	Very Low	Negligible	Metadata, Residuals

Statistical Modeling

- **Huffman/Arithmetic**: Use probabilities $P(X)$.
- **Neural**: Estimates $P(X)$ via non-linear density estimation.

The 1-bit Penalty

- **Huffman**: Fails if $H(X) < 1$ bit/symbol
- **Arithmetic**: Efficient below 1 bit/symbol

Context-Based vs. Block Coding

Experimental Data (T-shape image)

Method	Limit (bpp)	Practical (bpp)
Symbol-wise ($K = 1$)	0.586	1.000 (Huffman)
Block Coding ($K = 4$)	0.383	0.433 (Huffman)
Context-Based	**0.406**	$\approx$ **0.406** (Arithmetic)

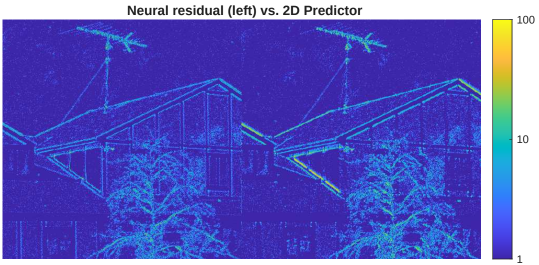
- **Block Coding**: Complexity grows exponentially (M^K)
- **Context-Based**: Reaches the entropy rate $\mathcal{H}$ by capturing the "significant past" without massive tables.

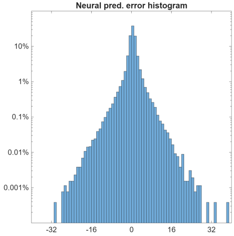
Neural Lossless Coding: SOTA Performance

Density Estimation Perspective

- NNs minimize the **Cross-Entropy** between the true source P and the model Q.
- They allows to have better predictors.

Experimental Validation (house.pgm):





The Neural Predictor (2.70 bpp) outperforms the 2D Predictor (2.83 bpp).

Lossless Design Matrix

If the source is...	Then use...	Why?
Memoryless	Huffman / Arithmetic	Respects $H(X)$ bound
Stationary w/ Memory	Context-Adaptive	Optimal for Markov stats
Locally Stationary	Adaptive Arithmetic	Tracks changing statistics
Highly Complex	Neural Models	Captures non-linear dependencies

Theoretical Convergence

All methods aim for $L \approx \mathcal{H}$. Choice depends on **complexity**, **latency**, and **model cost** (transmitting the NN weights).

Final Takeaway

Use **Huffman/EG** for speed, **Arithmetic/Context** for high compression, **Dictionary** for universal files, and **Neural Models** for the ultimate SOTA performance.